

NYC Airbnb Listings: From Exploratory Analysis to Predictive Modeling and Host Strategy Optimization

Shreyansh Misra

University of Massachusetts Amherst
shreyansh@umass.edu

Paransha Tajwani

University of Massachusetts Amherst
ptajwani@umass.edu

Abstract

We study the New York City Airbnb market using $\sim 48,895$ publicly available listings from Inside Airbnb, with two goals: predicting nightly price from public listing features, and turning those predictions into practical recommendations for prospective hosts. Our approach involved fitting a linear baseline, tuned tree-based regressors (HistGradientBoosting and Random Forest), a multilayer perceptron, and a stacked ensemble, trained on a winsorized log-price target.

The feature matrix joins the listings with the review-events table and three external sources: ACS census median household income, MTA subway stations, and four NYC anchor locations. Our final stacked ensemble achieves $R^2_{\log} = 0.6300$ and $\text{MAE} = \$47.25$ on a held-out 20% test set, a $+0.133$ improvement over the linear baseline ($R^2_{\log} = 0.4969$). SHAP identifies room type, the neighborhood-level price feature, and distance to Times Square as the dominant predictors, with ACS tract income contributing modest signal at rank 9.

We end with a grid search over actionable features that identifies the revenue-maximizing listing configuration for three prospective-host scenarios.

1 Introduction

New York City attracts more than 60 million tourists annually and hosts $\sim 48,000$ active Airbnb listings competing for guest nights across its five boroughs. For a prospective host, choices about how to list a property (whether to rent a single room or an entire unit, what minimum-night requirement to set, how aggressively to make the calendar available) directly determine nightly revenue. The publicly available data on existing listings, however, provides only price as an outcome (not booked nights or revenue) along with noisy features like unstructured listing names and uneven review activity. Recovering a meaningful fraction of price variance from these features, and translating the resulting model into actionable advice, is therefore a challenging problem.

- **RQ1:** What features of an Airbnb listing most strongly drive nightly price in NYC, and how much of the price variance can be explained from publicly available listing features?

- **RQ2:** Given a property in a specific NYC neighborhood, what listing configuration maximizes the predicted nightly price?

2 Related Work

Airbnb pricing analysis. Many studies now use Inside Airbnb data, originally compiled by housing advocate Murray Cox, to study short-term-rental pricing dynamics. Hill (Hill, 2015) created an early descriptive analysis of pricing in major US markets, and Wachsmuth and Weisler (Wachsmuth and Weisler, 2018) used the same data to study the impact of Airbnb on rental housing supply in NYC. More recent work has leveraged it for hedonic pricing studies (regressions of nightly price on listing attributes, location, and reviews) in cities including Barcelona, Sydney, and Berlin (Lawani et al., 2019; Gibbs et al., 2018). These studies consistently find that location and property type dominate nightly price, with review attributes adding small but consistent signal. Our exploratory analysis reproduces these findings on the NYC market.

Tabular machine learning. The shape of our modeling problem (tens of thousands of rows and tens of features, mixed numeric and categorical) means gradient-boosted decision trees are likely the strongest baseline on tabular benchmarks (Shwartz-Ziv and Armon, 2022). Comparisons of deep learning to gradient boosting on tabular data generally find that deep models match boosting only with substantial feature engineering, regularization, or architectural tuning, while boosting wins out of the box. We test this conventional wisdom by including a multilayer perceptron alongside our tuned tree models. Stacked ensembles, which combine heterogeneous base learners through a linear or shallow meta-learner, have been a dominant pattern in Kaggle competitions (Wolpert, 1992), and are an effective way to extract residual signal that any single model misses.

Interpretability. For tree-based models, SHAP values (Lundberg and Lee, 2017) are a standard method for per-prediction feature attribution. We use SHAP to explain the final tuned gradient-boosting model and to ground our scenario-based optimization in the same model that produced the predictive results.

3 Data

3.1 Source and overview

Our primary dataset comprises 48,895 NYC Airbnb listings from *Inside Airbnb* (Inside Airbnb, 2019), scraped from the public Airbnb website. Each listing carries 16 features covering nightly price, location (latitude, longitude, neighborhood, borough), room type, host identity, review counts, and availability. We also use the associated reviews table (991,000 rows) to derive a host-tenure feature. For the external features described in Section 3.7 we additionally join three public datasets: ACS 2015–2019 5-year median household income at the census-tract level (U.S. Census Bureau, 2020), the 2019 TIGER/Line census tract shapefiles for New York State (U.S. Census Bureau, 2019), and the MTA Subway Stations dataset from NY State Open Data (Metropolitan Transportation Authority, 2024).

3.2 Cleaning

Two issues required attention. First, $\sim 20.6\%$ of listings have missing review fields (`last_review`, `reviews_per_month`). These correspond to listings with zero reviews and were filled with zero rather than dropped. Second, 11 listings had a price of \$0, which we removed as data errors, leaving 48,884 listings.

3.3 Distributional properties

Price is heavily right-skewed (skewness = 19.12): the mean is \$153 but the median is only \$106, with a long tail topping out at \$10,000. Roughly 0.5% of listings list above \$1,000. Figure 1 makes the skew visible. This skew motivates two of our most important modeling decisions: training on $\log(1 + \text{price})$ and winsorizing price at the 99th percentile to cap the extreme tail.

3.4 Borough and room-type structure

Listings are concentrated in Manhattan (44.3%, mean price \$197) and Brooklyn (41.1%, mean \$124), with Queens, the Bronx, and Staten Island sharing the remaining 14.6%. Across 221 neighborhoods, room type splits into entire homes/apartments (52.0%, mean \$212), private rooms (45.7%, mean \$90), and shared rooms (2.4%, mean \$70).

3.5 Statistical structure

Numeric features show weak linear correlations with price (all $|r| < 0.1$), suggesting that price is driven by categorical and geographic factors rather than any numeric ones. ANOVA confirms that price differences across boroughs ($F = 354.6$, $p < 10^{-3}$) and across room types ($F = 1715.4$, $p < 10^{-3}$) are highly significant. Effect sizes (η^2) attribute 6.56% of price variance to room type and 2.82% to borough, making room type the stronger predictor.

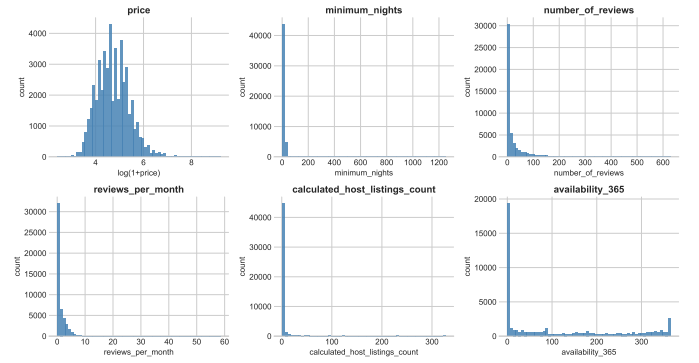


Figure 1: Distributions of key numeric features. Price exhibits extreme right skew (skewness = 19.12), motivating log-transformation and 99th-percentile winsorization for modeling.

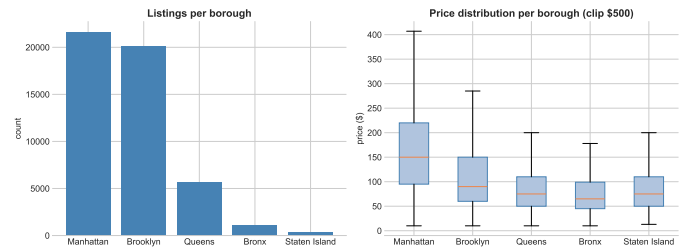


Figure 2: Borough-level listing counts and price distributions. Manhattan commands the highest prices despite Brooklyn having the most listings.

3.6 Key exploratory figures

Figure 1 shows the univariate distributions of key numeric features. Figure 2 shows borough-level listing counts and price distributions. Figure 3 visualizes the geographic price gradient. These figures motivated two specific engineered features: `dist_times_sq_km` (distance to a central tourism anchor) and `neighbourhood_median_price`.

3.7 External data integration

The Inside Airbnb dataset has many listing-side attributes but little information about the surrounding economic and transit context of each property. We therefore add three classes of external geographic data to each listing, chosen for their availability, interpretability, and *a priori* relevance to nightly pricing.

ACS census-tract median household income. For each listing we identify the corresponding 2019 census tract via spatial join against the TIGER/Line shapefile (U.S. Census Bureau, 2019) (Bronx, Kings, New York, Queens, and Richmond counties), then attach the American Community Survey 2015–2019 5-year estimate of median household income (table B19013, code B19013_001E) (U.S. Census Bureau, 2020). The 5-year vintage matches the 2019 Inside Airbnb scrape and is stable enough to be reliable. Among 48,884 listings (post-cleaning), the spatial join succeeds for 99.99%. About 0.5% of tracts have a sentinel income value (e.g. suppressed or non-residential) which we treat as miss-

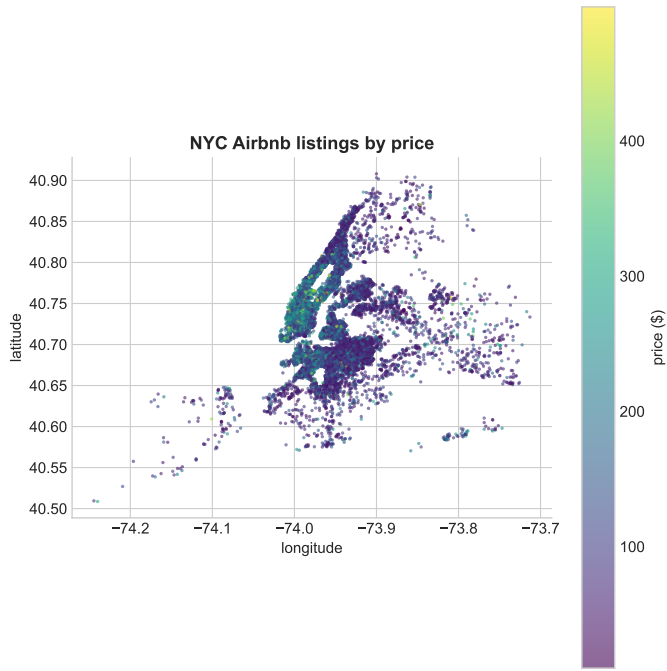


Figure 3: Listing locations colored by nightly price. The clear radial gradient centered on lower Manhattan motivates the distance-to-Times-Square engineered feature.

ing and impute with the training-set median. The resulting `tract_median_income` feature has mean \$86,276, ranges from \$11,167 to \$250,001, and exhibits strong within-borough variation that the Inside Airbnb neighbourhood variable does not capture.

MTA subway proximity. We pull the 472 active MTA subway stations from NY State Open Data (Metropolitan Transportation Authority, 2024) and compute the haversine distance from each listing to the nearest station. 78.4% of listings sit within 500 m of a station, and the median distance is 0.43 km. As an indicator of transit access, this feature tells us about the walkability and accessibility of an Airbnb listing.

Additional anchor distances. The original engineered feature set included only distance to Times Square as a tourism proxy. We add haversine distances to four additional NYC anchors: Central Park (40.7829°N, -73.9654°W), JFK Airport, LaGuardia Airport, and Wall Street. Multiple anchor distances let the model reason about location along several gradients simultaneously, rather than only having “distance from Times Square”.

In total, this integration adds six features: `tract_median_income`, `dist_subway_km`, `dist_central_park_km`, `dist_jfk_km`, `dist_lga_km`, and `dist_wall_st_km`.

3.8 Leakage-aware target encoding

The within-neighborhood median-price feature is derived from the target, so we construct it via out-of-fold target

encoding (Micci-Barreca, 2001). For training, the feature is built via 5-fold cross-validated median encoding. For each fold, the per-neighborhood median is computed on the other four folds and assigned to this fold’s rows, so no row’s feature ever depends on its own price. Test rows are encoded using neighborhood medians computed on the entire training set. Neighborhoods unseen in a given training fold fall back to the overall training median (< 0.1% of cases).

4 Methods

We frame price prediction as a supervised regression problem on the 48,884 cleaned listings, using a single 80%/20% train/test split with `random_state=42`, yielding 39,107 training rows and 9,777 test rows. All cross-validation reported here is 5-fold on the training set.

4.1 Target and feature engineering

Target transform. We train against $\log(1 + \text{price}_{\text{winsor}})$, where $\text{price}_{\text{winsor}}$ is the listing price clipped at the 99th percentile ($\sim \$500$). This caps the 0.5% of listings above \$1,000 without dropping them, so the loss is no longer dominated by a few luxury outliers. Dollar-scale metrics are reported by inverting the transform via `np.expml`. R^2 is reported in log-space, where the training loss is computed.

Engineered features. On top of the seven raw numeric features and the two categorical features (borough, room type), we add seven derived features and six external features (Section 3.7). The within-dataset features are:

- `dist_times_sq_km`: haversine distance from the listing to Times Square (40.7580°N, -73.9855°W).
- `host_tenure_days`: days between the host’s earliest review across any of their listings and the dataset’s last review date, as a proxy for host experience.
- `availability_ratio`: `availability_365/365`, the fraction of the year the listing is available.
- `name_length`: length of the listing name in characters, a rough effort signal.
- `neighbourhood_median_price`: out-of-fold target-encoded median price within the listing’s neighborhood (Section 3.8).
- `neighbourhood_listing_count`: count of listings in the same neighborhood, a density proxy.
- `reviews_per_tenure_day`: total reviews divided by host tenure, normalizing host activity.

The six external features are `tract_median_income`, `dist_subway_km`, `dist_central_park_km`, `dist_jfk_km`, `dist_lga_km`, and `dist_wall_st_km`,

as described in Section 3.7. After one-hot encoding of borough and room type, the within-dataset feature matrix has 20 columns. The full feature matrix with the six external features added has 26 columns.

4.2 Linear baseline

A linear-regression baseline trained on the seven raw numeric features and one-hot borough and room type, with $\log(1 + \text{price})$ as the target, serves as the floor against which all later models are compared. We also fit the same model on the engineered feature matrix to measure the gain from feature engineering alone.

4.3 Tree-based regressors and tuning

We train HistGradientBoosting (Shwartz-Ziv and Armon, 2022) and Random Forest regressors on the engineered feature matrix and the winsorized log-price target. Default-hyperparameter variants (HistGradientBoosting with `max_iter=400`, learning rate 0.06, 63 leaves, and Random Forest with 200 trees, `min_samples_leaf=3`) establish the strength of the tree-model family before tuning. We then tune both with Optuna (Akiba et al., 2019) using the TPE sampler: 40 trials over HistGB hyperparameters (learning rate, `max_iter`, `max_leaf_nodes`, `min_samples_leaf`, `l2_regularization`, `max_depth`) and 20 trials over Random Forest hyperparameters (`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`). The Optuna objective is the mean $\log\text{-}R^2$ across a 3-fold inner CV. The resulting best hyperparameters are then re-evaluated under the same 5-fold CV used for all other models, and reported in Section 5.

4.4 Multilayer perceptron

We train a PyTorch multilayer perceptron with three hidden layers of width (128, 64, 32), ReLU activations, dropout of 0.2 between layers, and a linear output. The network is trained against MSE on standardized inputs (per-fold StandardScaler) using AdamW ($\text{lr} = 2 \times 10^{-3}$, weight decay 10^{-4}), batch size 512, for up to 80 epochs with patience-10 early stopping on a held-out validation slice. The MLP is included as a comparison point since the literature consistently finds that on tabular problems of this size, well-tuned gradient boosting typically matches or beats a generic MLP, and we verify whether that pattern holds on our dataset.

4.5 Stacked ensemble

We stack the tuned HistGB, tuned Random Forest, and the MLP using a Ridge meta-learner ($\alpha = 1.0$). Out-of-fold predictions on the training set are produced via a 5-fold inner CV, so the meta-learner is trained only on predictions the base models did not see during their own training. The meta-learner is then evaluated under 5-fold CV.

4.6 SHAP interpretability

For the best tree model we compute SHAP TreeExplainer values on a 2,000-listing sample of the test set. These give per-prediction feature attributions, which we summarize via a bar chart of mean $|\text{SHAP}|$.

4.7 Effect of external features

To quantify how much the six external features add (Section 3.7), we refit the stacked-ensemble pipeline under cross-validation on two feature matrices: the within-dataset 20-feature matrix, and the 26-feature matrix with the external features added. Both runs share the same hyperparameters and modeling architecture, so the only varying factor is the feature matrix.

5 Results and Discussion

5.1 Cross-validation comparison

Table 1 reports the 5-fold CV performance of every model on the engineered + external feature matrix. Figure 4 visualizes the same comparison.

5.2 Best hyperparameters and stacking weights

Optuna’s best HistGB configuration is `learning_rate=0.053`, `max_iter=542`, `max_leaf_nodes=63`, `min_samples_leaf=6`, `l2_regularization=0.017`, `max_depth=9`. The best Random Forest is `n_estimators=382`, `max_depth=21`, `min_samples_leaf=3`, `max_features=0.557`. The Ridge meta-learner places weights (0.456, 0.568, -0.014) on (HGB, RF, MLP). The MLP’s coefficient is effectively zero, indicating that the stacked ensemble’s residual signal comes entirely from the two tree learners.

5.3 Held-out test performance

Table 2 reports test-set metrics for the four strongest models. The stacked ensemble achieves $R_{\log}^2 = 0.6300$ and $\text{MAE} = \$47.25$, the best test-set performance in the pipeline. Compared with the linear baseline on raw features ($R_{\log}^2 = 0.4969$, $\text{MAE} = \$54.66$), this is a +0.133 improvement in $R_{\log}^2$ and a $-\$7.41$ reduction in MAE.

5.4 SHAP interpretability

Figure 6 shows the mean- $|\text{SHAP}|$ ranking for the tuned HistGB on the full feature matrix. The dominant feature by a wide margin is `room_type_Private room` ($|\text{SHAP}|=0.36$), followed by `neighbourhood_median_price` (0.11), `availability_365` (0.10), `dist_times_sq_km` (0.07), and `minimum_nights` (0.07). Among the externally-derived features, `tract_median_income` enters at rank 9 (0.029) and `dist_wall_st_km` at rank 10 (0.026), with the remaining external features (subway proximity, distances to Central Park, JFK, and LaGuardia) farther down. The

Model	RMSE (\$)	MAE (\$)	$R^2_{\log}$
Linear (raw features only)	104.15	54.66	0.4969
Linear Regression	99.41 ± 1.51	51.89 ± 0.75	0.5636 ± 0.009
Ridge ($\alpha=1$)	99.45 ± 1.53	51.93 ± 0.77	0.5628 ± 0.009
Random Forest (default)	91.93 ± 0.94	47.43 ± 0.39	0.6297 ± 0.006
HistGradientBoosting (default)	91.87 ± 0.88	47.52 ± 0.43	0.6328 ± 0.006
Random Forest (tuned)	91.68 ± 0.96	47.12 ± 0.38	0.6351 ± 0.005
HistGradientBoosting (tuned)	91.65 ± 0.80	47.49 ± 0.44	0.6328 ± 0.006
MLP	111.22 ± 29.99	50.22 ± 1.64	0.5949 ± 0.018
Stacked (HGB+RF+MLP, Ridge)	91.22 ± 0.87	46.98 ± 0.40	0.6390 ± 0.006

Table 1: 5-fold CV performance across the full pipeline. The first row uses only the seven raw numeric features and the borough/room one-hots, and all other rows use the full engineered + external feature matrix. Dollar metrics are computed by inverting the log transform with $\exp m1$. R^2 is reported on the $\log(1 + \text{price}_{\text{winsor}})$ target. Mean $\pm$ std across folds.

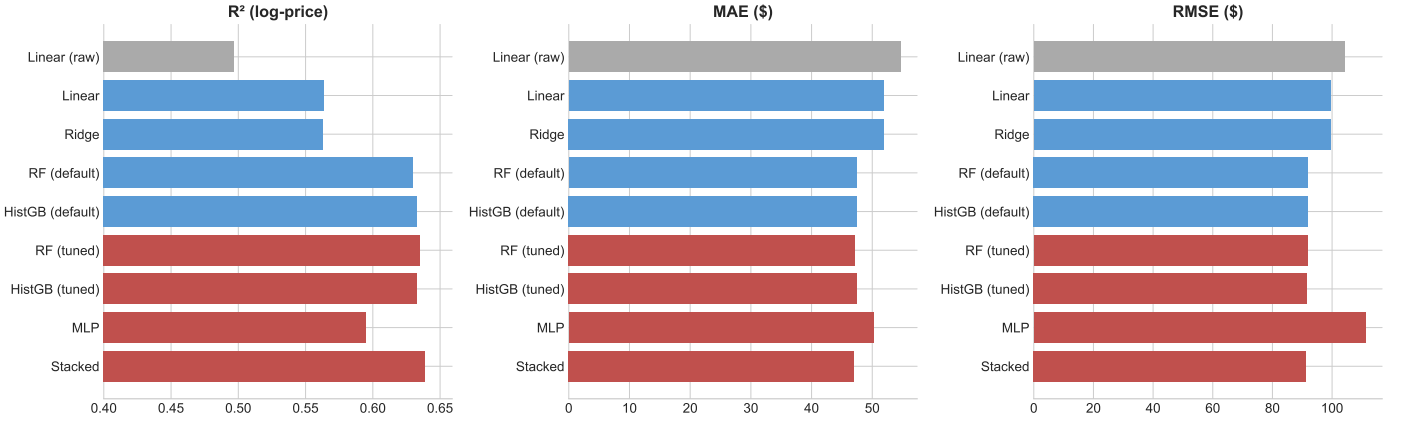


Figure 4: Model comparison. The largest single jump is from linear models to tree-based models, with tuning, MLP, and stacking producing a smaller but consistent further improvement.

Model	RMSE	MAE	$R^2_{\log}$	R^2_{raw}
Tuned HistGB	93.08	47.74	0.6255	0.4154
Tuned RF	92.96	47.40	0.6249	0.4170
MLP	98.53	50.79	0.5800	0.3454
Stacked (HGB+RF+MLP)	92.59	47.25	0.6300	0.4214

Table 2: Held-out test-set performance on the engineered + external feature matrix. Dollar columns are in raw \$, and R^2 is reported in both log and raw space.

directionality is intuitive throughout: private and shared rooms push price down, neighborhoods with high median price push it up, listings closer to Times Square or Wall Street push up, and listings in higher-income census tracts push up.

5.5 Residual diagnostics

Even with tuning, stacking, and the MLP, the residual signature visible at the tree-model stage persists. Predictions remain biased upward (i.e. the model under-predicts) for Manhattan and entire-home listings (Figure 5). The luxury tail is the hard part of this market for any model that uses only generic listing features and no booking-side data. We revisit this in Section 7.

Feature matrix	$R^2_{\log}$ CV	MAE CV
Without external (20 features)	0.6375	\$47.09
With external (26 features)	0.6390	\$46.98

Table 3: Stacked-ensemble 5-fold CV performance with and without the six external features. The lift from adding them is $\Delta R^2_{\log} = +0.0015$ and $-\$0.10$ in MAE.

5.6 Effect of external features

Table 3 and Figure 7 compare the stacked ensemble’s cross-validated performance with and without the six external features (Section 3.7), holding all other hyperparameters and architecture fixed.

Adding external geographic features moves the needle very little. Layering ACS tract income, subway proximity, and four NYC anchor distances on top of the within-dataset feature matrix yields $\Delta R^2_{\log} = +0.0015$ in CV and essentially no movement on the held-out test set. The within-dataset neighbourhood_median_price feature already captures most of the geographically-derivable signal at the resolution our model can exploit, so adding more geographic context buys very little. The simpler linear models do gain marginally more from the external features ($\Delta R^2_{\log} \approx +0.006$ for Linear/Ridge) than the tree models,

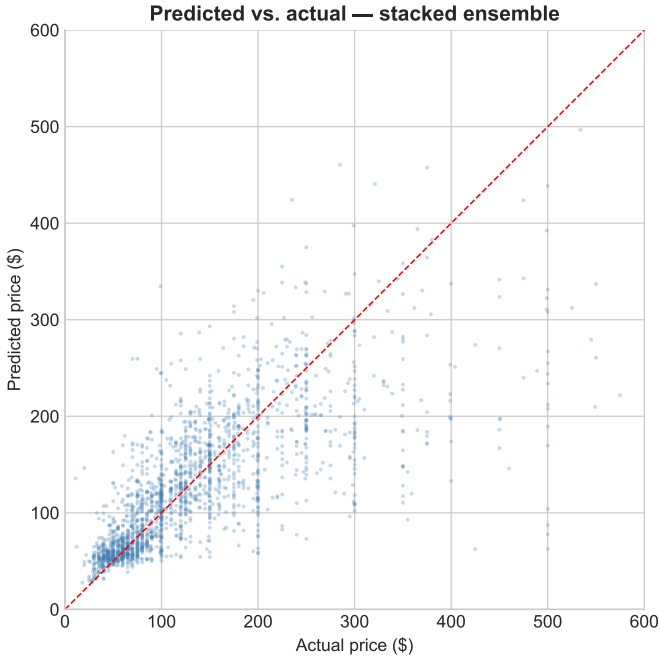


Figure 5: Predicted vs. actual price on the held-out test set for the stacked ensemble. Predictions stay tight on the diagonal up to $\sim \$300$, then progressively under-predict the luxury tail.

consistent with high-capacity trees already extracting from latitude and longitude what linear models cannot.

6 Optimal Listing Strategy

The predictive model from Section 5 answers RQ1. Translating it into actionable advice (RQ2) requires turning the model around: rather than predicting price for an existing listing, we ask, given a fixed location and a prospective host’s situation, which configuration of *actionable* features maximizes the predicted price. Actionable here means features the host can choose or change: room type, minimum-night requirement, calendar availability, and listing-name length. Static features (geography, neighborhood-level statistics, host tenure for new hosts) are held fixed at the scenario’s defaults.

6.1 Scenarios and grid

We define three prospective-host scenarios that span the dominant borough/room-type segments visible in the exploratory analysis:

- **Studio in Williamsburg** (Brooklyn): the host owns a studio apartment and is choosing between renting it as an entire unit or as a private room.
- **Spare room in Astoria** (Queens): the host has a private or shared room available in their primary residence.
- **Entire apt in Harlem** (Manhattan): the host has full control of an apartment in upper Manhattan.

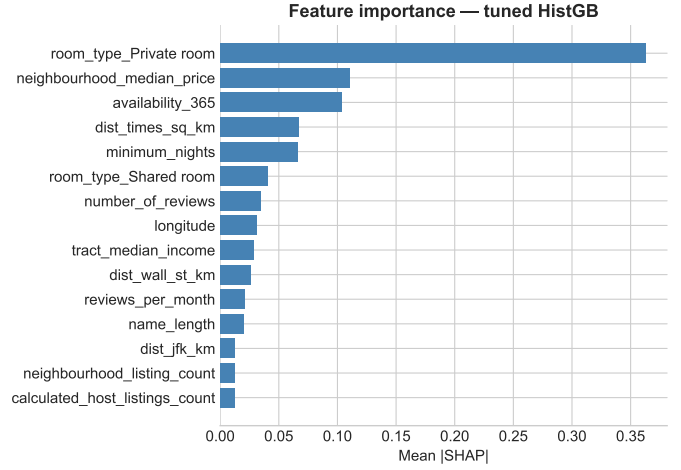


Figure 6: Mean |SHAP| ranking on a 2,000-listing sample of the test set, tuned HistGB on the full feature matrix.

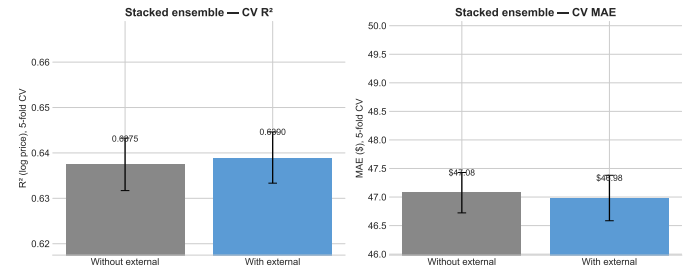


Figure 7: Stacked-ensemble CV $R^2_{\log}$ and MAE with and without external features, identical hyperparameters. The lift from adding them is small in absolute terms.

For each scenario, latitude, longitude, neighbourhood_median_price, and neighbourhood_listing_count are pinned to the neighborhood’s actual values, and host attributes are set to those of a new host (zero reviews, zero tenure, single listing). We then grid over actionable features: $\text{minimum_nights} \in \{1, 2, 3, 7, 30\}$, $\text{availability_365} \in \{60, 180, 270, 350\}$, $\text{name_length} \in \{25, 50, 80\}$, and the room-type set appropriate to the scenario.

6.2 Recommendations

Table 4 reports the predicted-price-maximizing configuration for each scenario, and Figure 8 shows the top-10 configurations per scenario.

A few patterns emerge consistently:

- **Whole-unit framing wins** where the host has the option. SHAP confirms room type is the single largest contributor to predicted price.
- **Higher availability is preferred** over very low availability (which signals an under-committed listing), but extreme availability (350+ days) buys little additional predicted price beyond the mid-range.
- **Longer listing names** are weakly associated with

Scenario	Optimal config	Price (\$)
Studio in Williamsburg	Entire home/apt, min nights 2, avail. 350 d, name length 80	402
Spare room in Astoria	Private room, min nights 2, avail. 350 d, name length 80	126
Entire apt in Harlem	Entire home/apt, min nights 3, avail. 350 d, name length 80	289

Table 4: Predicted-price-maximizing configuration per scenario, found by grid search through the tuned HistGB model.

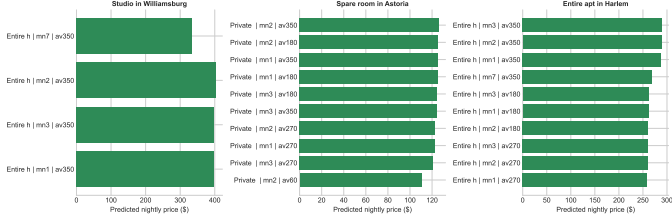


Figure 8: Top-10 configurations per scenario by predicted nightly price. Patterns are consistent across scenarios: entire-home framing (where available) wins, high availability wins, and longer listing names correlate with higher predicted price.

higher predicted price, consistent with the proxy interpretation that effort put into the listing helps. The effect is small but consistent.

- **Minimum-night requirements** have a non-trivial optimum: long minimum stays (30+ nights) push predicted price down across all scenarios, consistent with the longer-stay market being a different and lower-priced segment.

In the Williamsburg scenario, for instance, the model’s recommended configuration (Entire home/apt, min nights 2, availability 350 d, name length 80) yields a predicted nightly price of \$402, roughly \$252 above the neighborhood’s median.

7 Limitations and Future Work

No booking data. Inside Airbnb provides listed *prices*, not bookings or revenue. The optimization in Section 6 therefore maximizes predicted nightly price, which is a proxy for, but not the same as, host revenue. A listing priced at \$300 a night that books 50% of available nights earns less than one priced at \$200 a night that books 80%. Future work should incorporate either explicit booking data (where legally available) or an occupancy-prediction model trained on review-rate proxies.

Persistent luxury-tail bias. Predictions remain biased upward (the model under-predicts) for Manhattan and entire-home listings, even after tuning, the MLP, and stacking. We hypothesize that this reflects information not captured in the public features: listing photos, amenity lists, and listing-description text, all of which are public on

Airbnb but not in the Inside Airbnb scrape. A multimodal model incorporating image and text features would be a natural extension.

External geographic features add little. Our comparison (Section 5.6) shows that adding ACS tract income, subway proximity, and four NYC anchor distances on top of the within-dataset feature matrix produces almost no lift ($\Delta R^2_{\log} = +0.0015$ in CV, zero on test). The rate-limiting constraint on this problem is not geographic context but per-listing variation unobserved in the structured data, particularly listing-description text, amenities, and photos. Future work should prioritize these data sources, or booking-side data, over additional geographic features.

Static snapshot and validation split. Our data is a single 2019-era snapshot, so seasonality and the impact of NYC’s Local Law 18 short-term-rental restrictions are out of scope. Our 80/20 split is also random rather than geographic. A geography-stratified split would be a stronger test of whether the model generalizes to neighborhoods it has not seen.

8 Conclusion

We built a price-prediction pipeline for NYC Airbnb listings combining tuned HistGradientBoosting and Random Forest models with an MLP and a Ridge-stacked ensemble, trained on a feature matrix that joins the Inside Airbnb listings with the host-tenure derived from the review events table and external data from the ACS, the MTA, and a small set of NYC anchor locations. The within-neighborhood median-price feature uses out-of-fold target encoding. The full pipeline achieves $R^2_{\log} = 0.6300$ and MAE=\$47.25 on a held-out test set, a +0.133 improvement in $R^2_{\log}$ and a −\$7.41 reduction in MAE relative to a linear baseline on raw features. SHAP attributions identify room

type, the neighborhood-level price feature, and distance to Times Square as the dominant predictors, with ACS tract income contributing modest signal at rank 9. Removing the six external features changes cross-validated $R^2_{\log}$ by only −0.0015, locating the remaining frontier of this problem outside structured geographic data. The scenario-

based optimization shows that the predictive model carries enough signal to make actionable recommendations to a prospective host: across three scenarios spanning Brooklyn, Queens, and Manhattan, the revenue-maximizing configuration consistently prefers whole-unit framing (when available), short minimum stays, high availability, and longer listing names.

References

- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631.
- Chris Gibbs, Daniel Guttentag, Ulrike Gretzel, Jym Morton, and Alasdair Goodwill. 2018. Pricing in the sharing economy: A hedonic pricing model applied to Airbnb listings. *Journal of Travel and Tourism Marketing*, 35(1):46–56.
- Dan Hill. 2015. How much is your spare room worth? *IEEE Spectrum*, 52(9):32–58.
- Inside Airbnb. 2019. Get the data: Adding data to the debate. <https://insideairbnb.com/get-the-data>. NYC scrape; accessed 2026-03-03.
- Abdul Lawani, Michael R. Reed, Tyler Mark, and Yuqing Zheng. 2019. Reviews and price on online platforms: Evidence from sentiment analysis of airbnb reviews in Boston. *Regional Science and Urban Economics*, 75:22–34.
- Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 4765–4774.
- Metropolitan Transportation Authority. 2024. MTA Subway Stations. NY State Open Data, dataset 39hk-dx4f. <https://data.ny.gov/Transportation/MTA-Subway-Stations/39hk-dx4f>.
- Daniele Micci-Barreca. 2001. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Ravid Shwartz-Ziv and Amitai Armon. 2022. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90.
- U.S. Census Bureau. 2019. TIGER/Line Shapefiles, 2019: Census Tracts, New York State. <https://www2.census.gov/geo/tiger/TIGER2019/TRACT/>.
- U.S. Census Bureau. 2020. American Community Survey 5-Year Estimates, 2015–2019, Table B19013 (Median Household Income in the Past 12 Months). <https://api.census.gov/data/2019/acs/acs5>.
- David Wachsmuth and Alexander Weisler. 2018. Airbnb and the rent gap: Gentrification through the sharing economy. *Environment and Planning A: Economy and Space*, 50(6):1147–1170.
- David H. Wolpert. 1992. Stacked generalization. *Neural Networks*, 5(2):241–259.